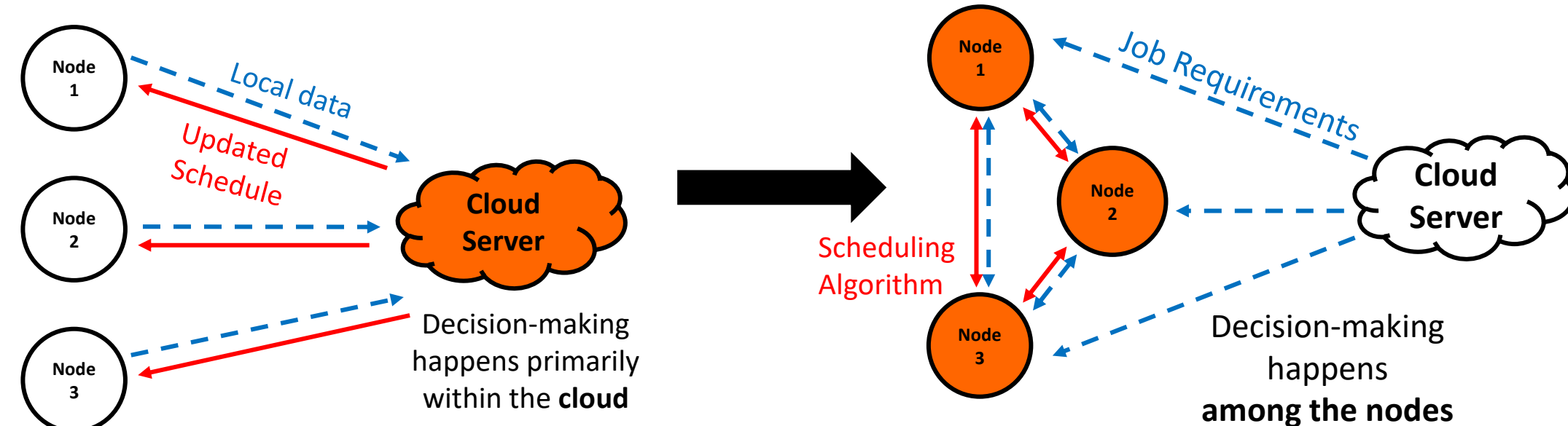


Introduction

In recent years, edge computing has brought new opportunities to move computation away from the cloud and to the edge, accelerating reaction times to local changes for time-sensitive applications. However, task allocation in this setting can be challenging due to changes in node capabilities, the need to offload tasks from busy nodes to free nodes, communication constraints, and dynamically shifting task priorities on a per node and per network basis. We propose to use a distributed optimization framework to allow the system to find the desired allocation of tasks across the network without use of the cloud.



Algorithm

We propose using a standard dual descent method to solve the task allocation problem, shown below, with G and $\mathbf{q}$ being the vertical concatenation of $(\mathbf{g}_m \otimes \mathbf{e}_t)^T$ and q_m .

1. Initialize $\mathbf{x} = 0, \lambda = 0, k = 0, \alpha > 0$.
2. **for** $k \geq 0$ **do**
3. $\mathbf{x}(k+1) = \operatorname{argmax}_{\mathbf{x} \in \mathcal{X}} \sum_{i \in \mathcal{N}} U_i(\mathbf{x}_i) + \lambda(k)^T G \mathbf{x}$
4. $\lambda(k+1) = \max\{\lambda(k) - \alpha(G \mathbf{x}(k+1) - \mathbf{q}), 0\}$
5. **end for**

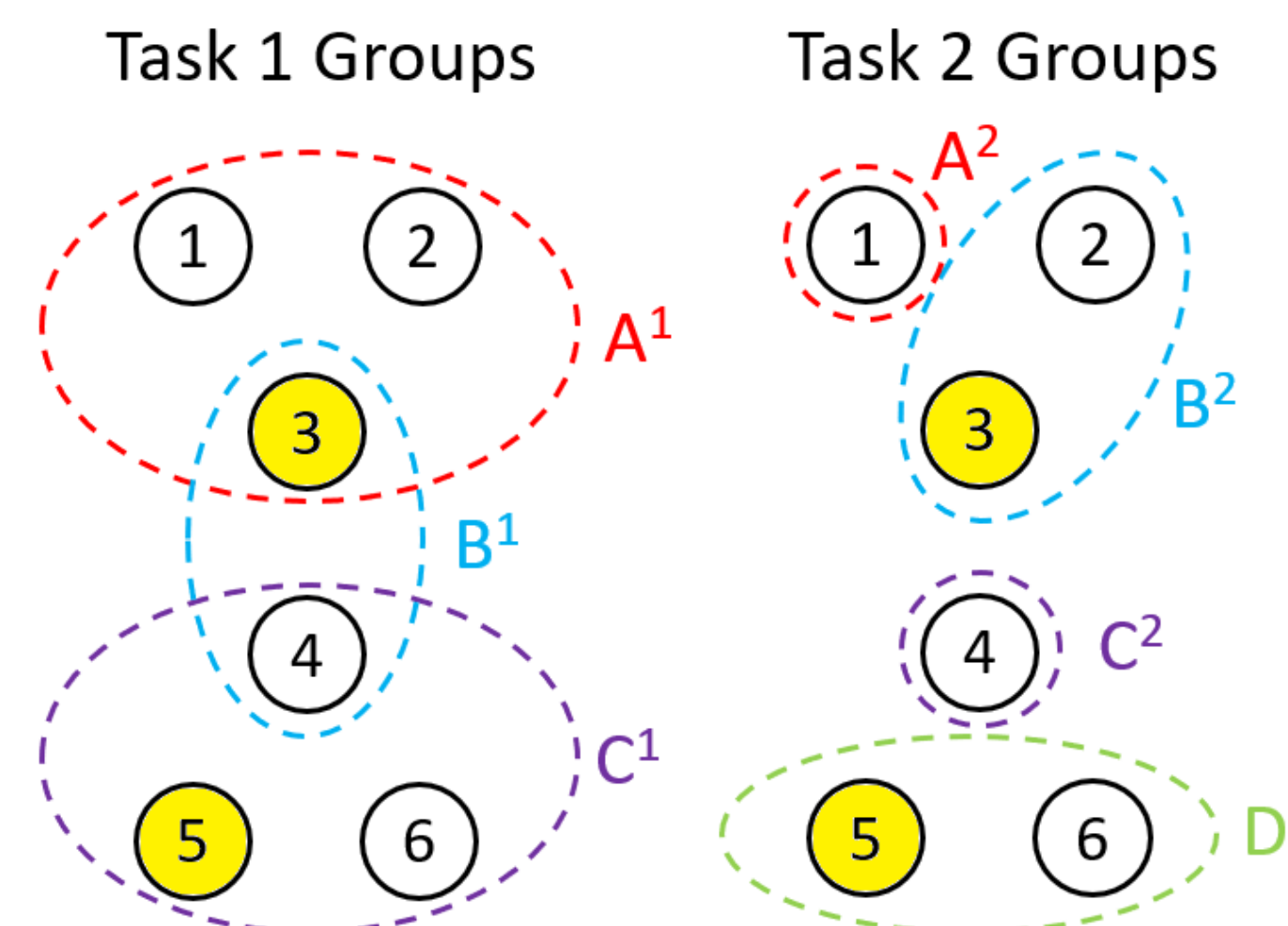
Dual-descent can be performed without issue under our communication constraints, and assuming concave utilities $U_i()$, the algorithm converges to the optimum policy by Slater's Condition.

Problem Formulation

Let $\mathcal{N}$ be the set of nodes (possibly heterogeneous), and let N be the total number of nodes in the network. Let $\mathcal{T}$ be the set of tasks, and let T be the total number of tasks. Every node belongs to one or more node groups $m \in \mathcal{M}$ associated with a task t which each represents a set of nodes that are in some way able to meaningfully cooperate on a given task. We represent each node group individually m via the vector $\mathbf{g}_m \in \mathbb{R}^N$. $\mathbf{g}_m[i] > 0$ if node i is included in the i th node group, and $\mathbf{g}_m[i] = 0$ otherwise. We assume that nodes are able to communicate freely with other nodes in the same node group.

We want to find a resource allocation vector $\mathbf{x} = [\mathbf{x}_1 \dots \mathbf{x}_N]^T \in \mathbb{R}^{NT}$ that optimizes over the sum of each node's utility, $U_i(\mathbf{x}_i)$, subject to the hardware constraints of each node $\mathcal{X}_i$ (jointly $\mathcal{X}$). Finally, we impose a QoS minimum per node group, q_m , such that the total effort allocated to t across m is given by $(\mathbf{g}_m \otimes \mathbf{e}_t)^T \mathbf{x}$, where $\mathbf{e}_t$ is the unit vector and $\otimes$ is the Kronecker product. All this yields the optimization problem:

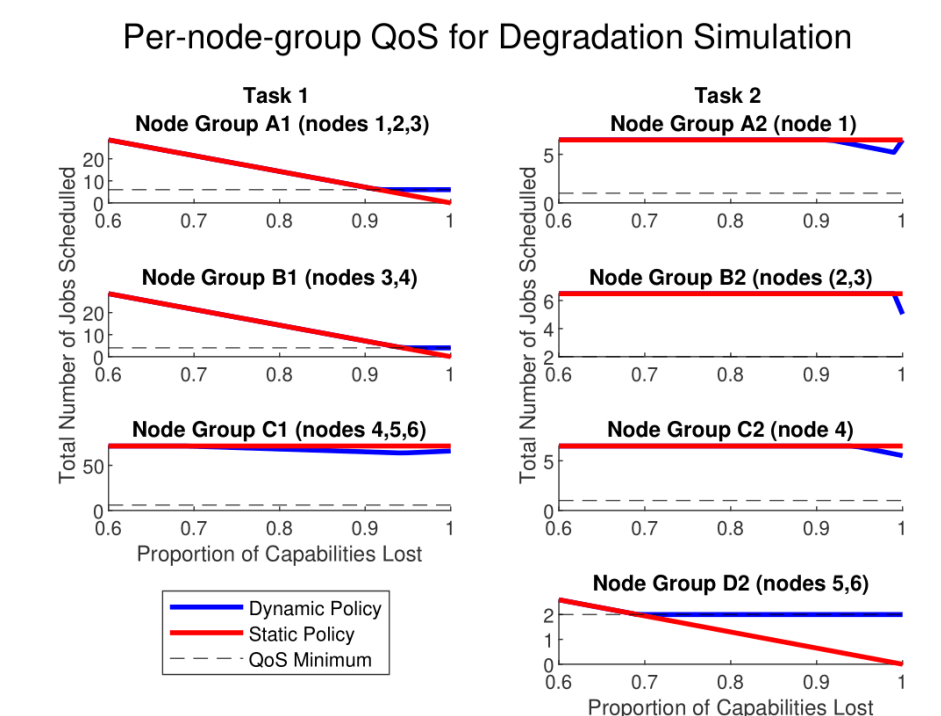
$$\max_{\mathbf{x} \in \mathcal{X}} \sum_{i \in \mathcal{N}} U_i(\mathbf{x}_i) \quad \text{s.t.} \quad (\mathbf{g}_m \otimes \mathbf{e}_t)^T \mathbf{x} \geq q_m \quad \forall m \in \mathcal{M}$$



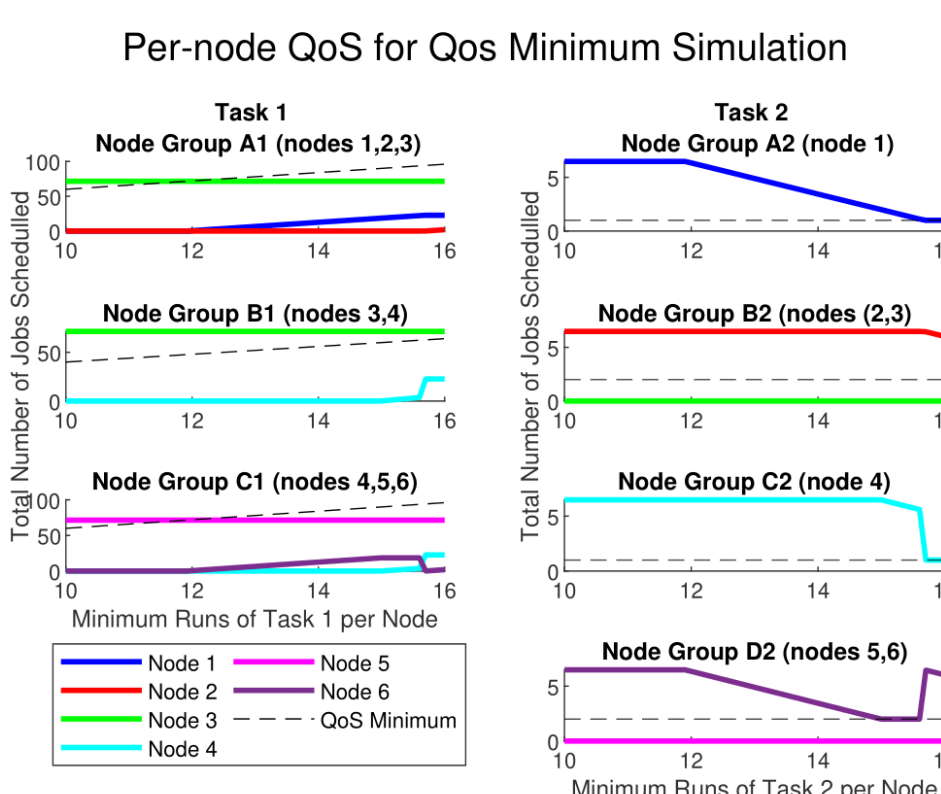
Numerical Simulation

To design a simulation that reflects one possible real setting for edge computing, we use ANL's **Sage** network's cyberinfrastructure to generate a dataset. We assumed the network structure as in the diagram in the problem formulation, with white nodes being **Raspberry Pi** units, and yellow being **Nvidia Jetsons**. $\mathcal{X}_i$ are constrained by both memory and power. Jobs run on a Jetson yield higher utilities than jobs run on a Raspberry Pi. The network runs two tasks. We collected data from runs of these tasks on actual devices to get the power and memory requirements of each task. Based on the requested frequency of running those tasks, we then derived the per-cycle QoS minima.

Comparing against a static policy, we continually degraded the memory & power cap for nodes 3 & 6 from its baseline all the way down to 0. The dual-descent method is able to cope with the changes and reallocate jobs to still meet the QoS minimum.



We also ran a simulation that varied the QoS minimum of task 1 for each node group between 2 (baseline) to 16. Once again, the dual-descent method maintained the now changing QoS minimum while the static policy failed.



Conclusion

We present a novel framework for task allocation in edge computing which can capture node heterogeneity and different dynamic attributes of the network's nodes and tasks. Future work includes verification on real hardware, and improving the flexibility of the framework by using a more general utility.